Exercises

Exercise 6: Graphics for communication

Read Chapter 4, and Section 4.3 in particular, to help you complete the questions in this exercise.

Everything you plotted in Exercise 5 was for **you**. Exploratory plots are quick, ugly and disposable: their only job is to show you what is in your data, and it does not matter that the axis label says `cardiac$tchol` because you are the only person who will ever see it. This exercise is about the other kind of plot, the one that goes on a poster or in a report, where the reader is not you, has about fifteen seconds, and will not ask you what the axis means. Nearly all of the work is in the details you skipped last week.

You will also switch datasets. From here on the course uses data from the Scottish Public Health Observatory, which is the same source you will use for your own assessment.

1. As in previous exercises, either create a new R script or continue with your previous R script in your RStudio Project. Again, make sure you include any metadata you feel is appropriate (title, description of task, date of creation etc) and don't forget to comment out your metadata with a `#` at the beginning of the line.

2. Download the data file *'scotpho_alcohol_admissions.txt'* from the **Data** link and save it to your `data` directory. This one is already a tab delimited text file, so there is no conversion to do. Import it with `read.table()`, remembering to write out your `header`, `sep`, `na.strings` and `stringsAsFactors` arguments, and assign it to a variable called `scotpho`.

3. These data are an extract from the ScotPHO online profiles tool. The indicator is *alcohol-related hospital admissions*, and the `measure` column is an age-sex standardised rate per 100,000 population: in other words, how many people per 100,000 were admitted to hospital for a reason related to alcohol, adjusted so that areas with different age and sex profiles can be compared fairly. The extract covers each of the 32 Scottish council areas plus Scotland as a whole (`area_name`, `area_code`, `area_type`), for each year from 2010 to 2019 (`year`). Because the rate is an estimate rather than a count, it comes with a confidence interval (`lower_confidence_interval`, `upper_confidence_interval`), which gives you a sense of how precise the estimate is.

4. Before plotting anything, get your bearings, exactly as you did in Exercise 3. Use `str()` and `summary()` on `scotpho`. How many rows are there? Use `table()` on `area_type` to see how the rows split between council areas and Scotland. Then check something that matters a great deal before you draw a line through points: does every area have a value for every year? (hint: `table()` on `area_name` will tell you how many rows each area has). What would your plot look like if one area were missing 2015?

5. Start with the simplest possible communication plot: how have alcohol-related admissions across Scotland as a whole changed over the ten years? Extract just the rows where `area_name` is "Scotland" and use `plot()` to plot `measure` on the y axis against `year` on the x axis. Use the `type` argument to draw both points and a line joining them. Now look hard at what R has given you by default, and list everything a stranger would not understand about it.

6. A single line rarely tells the whole story. Scotland's average hides enormous variation between council areas. Add two council areas to your plot: "Glasgow City", which has the highest rate in Scotland, and "Aberdeenshire", which has one of the lowest. Pull each area out into its own dataframe first, exactly as you did with Scotland. Then plot one of them with `plot()` and add the other two on top using the `lines()` function. Two things will catch you out. First, `plot()` sets the axis limits from whatever you plot first, so if you start with Scotland the Glasgow line will run off the top of the plot; use the `ylim` argument to set limits that fit all three. Second, give each area its own colour, and add a `legend()` so the reader knows which line is which.

7. Now think about those colours. Roughly 1 in 12 men has some form of colour vision deficiency, most commonly difficulty distinguishing red from green - and red and blue are not a safe pairing either. Posters also get photocopied, printed in black and white, and viewed on projectors that wash colour out. Redraw your plot using a colour-blind friendly palette: base R has one built in, `palette.colors(3, palette = "Okabe-Ito")`. Then test it. Redraw it a third time replacing those colours with their greyscale equivalents, which are `grey(c(0, 0.64, 0.62))`, and see whether the three lines are still tellable apart. If they are not, what could you change other than the colour?

8. Time to fix the words. Every plot that leaves your computer needs axis labels a non-specialist can read, including units; a title that says what the reader is looking at; and a note saying where the data came from. Write the labels for someone who has never heard the phrase 'age-sex standardised' - your reader is a public health colleague or a member of the public, not a statistician. Add the source underneath the plot using `mtext()`. Finally, think about the y axis. Should it start at zero? There is no single right answer, but you should be able to justify whichever you choose. Keep this version of the code safe, because you will need it again in Q9.

9. Finally, get the plot out of R at a size and resolution fit for a poster. Screen defaults are no use here: a plot that looks fine in the RStudio plot pane will be a blurry mess when blown up to A1. Save your finished plot twice, once as a pdf and once as a png, by putting the plotting code you wrote in Q8 between a `pdf()` or `png()` call and `dev.off()` (see Section 4.5). Use a 16:9 shape, which is what most poster and slide templates expect. For the png, set the resolution explicitly - 300 dpi is the usual minimum for print. Open both files and check them at full size before you believe them.

Optional alternative: the same plot using ggplot2

This section is entirely optional and is not assessed. Base R graphics are what this course teaches and what your assessment expects, and everything above is complete on its own. It is here because `ggplot2` is widely used and some of you will meet it elsewhere, so it is worth seeing the same figure built the other way.

Notice that `ggplot2` saves you the repetition - one `geom_line()` handles all three areas because it splits them by `area_name` - but it does not make a single one of the decisions for you. The palette, the redundant line type, the labels, the source note and the zero on the y axis are all still yours.

End of Exercise 6